

EIP-712 Signature Replay, Security Review

Target: `SignatureClaim`, a gasless EIP-712 claim vault **Type:** Demonstration review on intentionally vulnerable code **Method:** Foundry proof-of-concept, vulnerable branch (`master`) and remediated branch (`fixed`)

This report documents one finding. A passing proof-of-concept reproduces the exploit on `master`; the same scenario, run against the remediated contract on `fixed`, shows the attack neutralised.

H-01, Signature-malleability replay (High)

A relayer can pay any signed claim out twice. In the proof-of-concept the authorizer signs a single claim of 1,000 ether for `bob`; the relayer submits the signature and then its malleable twin, and `bob` receives 2,000 ether. The extra 1,000 ether is drained from the vault against one authorisation. The attack needs no special role and no interaction from the signer beyond producing one legitimate signature.

Root cause

The contract prevents replay by remembering the signature bytes:

```
bytes32 sigKey = keccak256(abi.encodePacked(r, s, v));
require(!usedSig[sigKey], "signature already used");
...
usedSig[sigKey] = true;
```

ECDSA signatures are malleable. For any valid (r, s, v) over a digest, the pair $(r, N - s, v \wedge 1)$, where N is the `secp256k1` group order, recovers the same signer for the same digest. The two encodings hash to different `sigKey` values, so the second one is seen as a fresh signature. Because the guard keys on the encoding rather than on the message, the same authorisation is accepted twice.

Attack

1. The authorizer signs `Claim(to = bob, amount = 1000e18)` off-chain, producing (v, r, s) with a canonical low `s`.
2. A relayer calls `claim(bob, 1000e18, v, r, s)`. `bob` receives 1,000 ether.
3. The relayer computes the twin: $s_2 = N - s, v_2 = v \wedge 1$. It recovers the same authorizer over the same digest.
4. The relayer calls `claim(bob, 1000e18, v_2, r, s_2)`. `usedSig` has no entry for the twin, so the check passes and `bob` receives another 1,000 ether.

Proof of concept

`test/SignatureClaim.poc.t.sol`, function `test_malleability_doubleSpend`, run on `master`:

```
authorizer signed (wei): 1000000000000000000000
bob received      (wei): 2000000000000000000000
net theft        (wei): 1000000000000000000000
```

Recommendation

Two changes, both standard practice:

1. Key the replay guard on the message digest, not on the signature bytes. Every encoding of one authorisation shares the same digest, so the second submission collides.
2. Enforce EIP-2: reject s in the upper half of the range and reject non-canonical v . This refuses the malleable twin before recovery.

```
uint256 private constant HALF_N =
    0x7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF5D576E7357A4501DDFE92F46681B20A0;

require(uint256(s) <= HALF_N, "malleable signature");
require(v == 27 || v == 28, "bad v");
...
require(!usedDigest[digest], "claim already executed");
usedDigest[digest] = true;
```

This is the behaviour of OpenZeppelin's ECDSA library. On the `fixed` branch the same proof-of-concept shows the malleable twin rejected by the low-s check and an exact replay rejected by digest tracking; bob receives exactly 1,000 ether.

Severity

High. The impact is direct theft of vault funds. The actor is any relayer, a role with no privilege, and the only precondition is observing one relayable signature.

Scope and disclaimer

SignatureClaim is intentionally vulnerable code written to demonstrate audit methodology end to end. It is not production code and must never be deployed. The finding above is a real vulnerability in this demo contract, reproduced with an executable proof-of-concept, not an invented severity.